

another and what the trade-offs are, in terms of efficiency.

Here is a question: If you want to be able to minimise the invalidation of iterators and pointers, while being able to provide transactional semantics for insertions and deletions, which STL container would you use and why? By this single question, the interviewer gets to know not only how much you know about STL containers, but also about your ability to think through multiple choices and trade-offs. By throwing in the part about transactional semantics, he is able to assess how comfortable you are with exception safety and concurrency as well.

Most interview questions are designed this way. They are expected to test not just what you know, but whether you can apply your knowledge to a different situation. More importantly, they test whether, given adequate information, you are able to use the information to build solutions to problems you may not have come across before. All of us are familiar with how to find the minimum in an array of integers. If the array is already sorted, it is an $O(1)$ operation to return the minimum. Now consider a slight variant of the question: Given a sorted array of integers that has been rotated an unknown number of times, give an $O(\log N)$ algorithm to find the minimum element.

When you are given a question, first think about other problems for which you know the solution, which are similar. In this case, we know that finding a minimum in a sorted array is a similar problem. Another related problem is finding a particular element in a sorted array. Next, think about whether you have been given any additional information that can help you narrow your choice down to a particular algorithm. In this case, the fact that you have been told that you need to design an $O(\log N)$ algorithm should make you think about using a binary search. Now that you have narrowed down the approach to a binary search, think about how you can modify a standard binary search to solve this problem. Since the sorted array has been rotated, you will have an ascending sequence of integers, followed by an inversion or reset point which marks the start of the original sorted array, and then the sequence of integers ascend again. Now the minimum is the reset point. Having arrived at this, it becomes a straightforward issue to code the solution.

To write efficient code, there is no way of preparing except to "Write code, test it, write more code, test it ..." and so on, ad infinitum. Whenever you come across a programming question, please make it a point not to look at the solution first. First write down your version of the solution, code it and test it. Then compare it with the given solution to see where the differences are.

Ask yourself the question, "Does my code handle all the kinds of data inputs that are expected?" For instance, the problem input may start with "Given an array of integers..." Am I making an assumption that the size of the array is less than some limit? What if there are more integers than can fit

in the main memory? Has any specification been given on the limits the values can have? For instance, if you are told that the integers represent age, then we know that they can be assumed to be limited between 0 and 100. But remember one thing: The only way you can become an expert in coding is to actually write code, test and run it. No amount of reading books on coding can prepare you for writing actual code. This is especially important for students who do not have an opportunity to write code every day. Please set aside some time every day and solve programming problems by writing actual code.

My 'must-read book' for this month

This month's 'must-read book' suggestion is not a single book, but a set of books which I believe are must-reads in order to develop expertise in programming. Some of them have already been mentioned in our column before.

- Introduction to Algorithms*—Cormen, Leiserson and Rivest
 - Programming Pearls*—Jon Bentley
 - Programming Interviews Exposed*—Mongan, Suojanen and Giguère
 - Cracking the Coding Interviews*—Gayle Laakmann
- For C++, I recommend Scott Meyers' 'Effective C++' and 'Effective STL' series once you have read a basic C++ book. For C, after reading Kernighan and Ritchie's 'C Programming Language', my favourite is 'Expert C Programming Deep C Secrets' by Peter Van der Linden. Get your basics in operating systems clear by reading the book 'Operating Systems' by Silberschatz and Galvin. For computer architecture, there is no better book than 'Computer Architecture: A Quantitative Approach' by Hennessy and Patterson.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in this column. It would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming, and here's wishing you the very best! 

By: Sandya Mannarswamy

The author is an expert in systems software and works at Hewlett-Packard India. Her interests include compilers, multicore technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=&gid=2339182>.